bool datatype:

We can use this datatype to represent boolean values
The allowed values for boolean data type are : True, False

is eligible -> Yes (True)
is not eligible --> No (False)

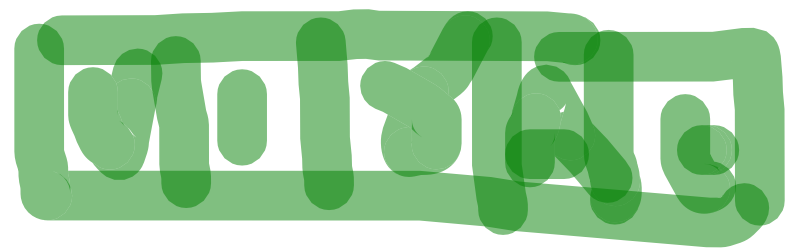
Is customer active --> yes or no --> True or False
a = True
status = True
is_eligible = True
is_active = True

internallly **True** means 1 and **False** means 0

string datatype :

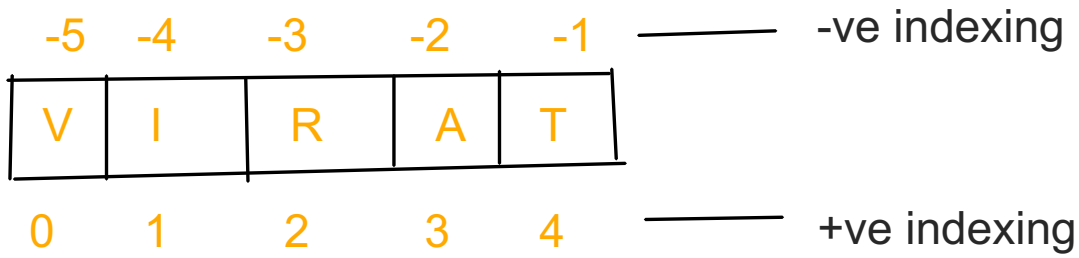
str is the representation of string datatype
A string is sequence of characters, enclosed within a single quotes or double quotes also
it can have a alphanumberic values as welll

favourite_cricketer = "virat"
customer_name = "dhoni MS"
order_info = "ordered selected"
email_id = "test@gmail.com"



"Python class is very and useful" for java students also'"

cricketer= "virat"



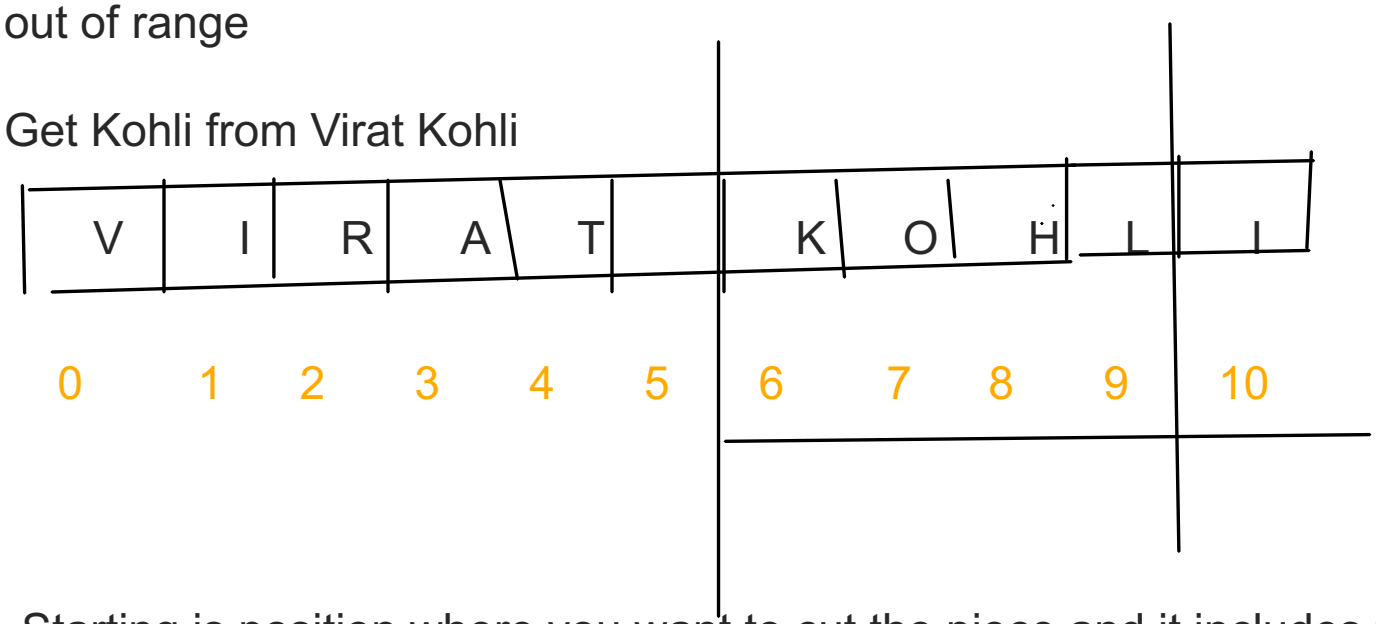
Slicing of Strings:
slice means a piece of big portion
[ ] is it s slicing operator, which can used to retrieve parts of string

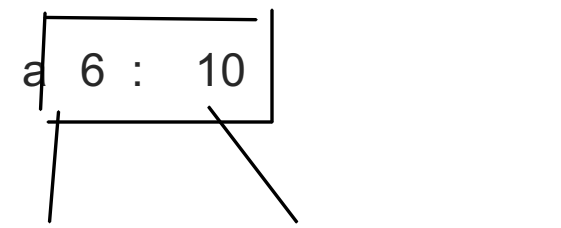
In python string follows indexing mechanism to store characters
Index always starts from zero (0,1,2,3,4,.....)

In python indexes can be postitve or negative
+ve index means forward direction from left to right
-ve index means backward direction from right to left

Note: if we try to fetch value using the index more than the indexed position then we get IndexError: string out of range

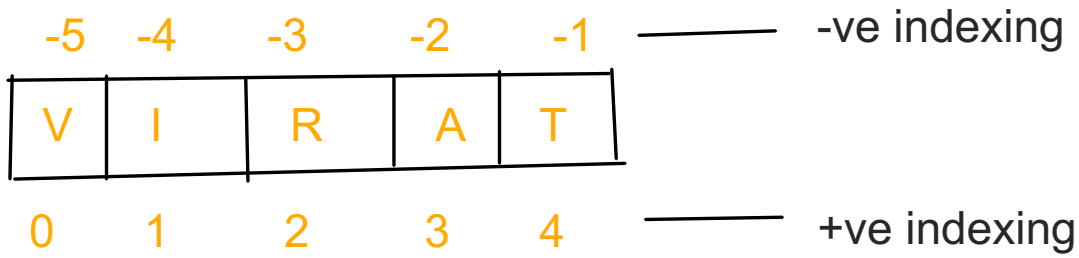
Get Kohli from Virat Kohli





Starting is position where you want to cut the piece and it includes that position character
endpoisition is where you want to end the piece but ending index **chracter wont by inclusive**
If we dont give any endposition, ending index will be treated as the end of entire string

cricketer= "virat"



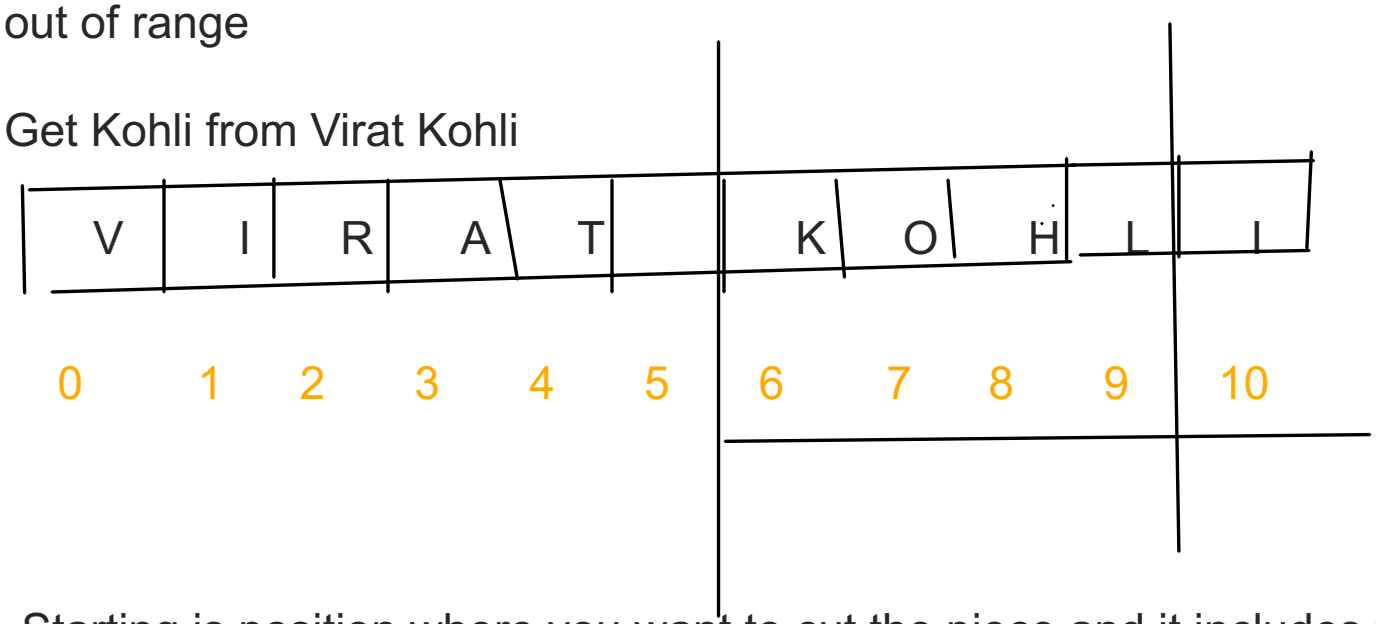
Slicing of Strings:
slice means a piece of big portion
[ ] is it s slicing operator, which can used to retrieve parts of string

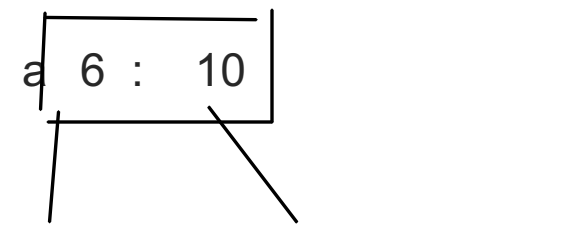
In python string follows indexing mechanism to store characters
Index always starts from zero (0,1,2,3,4,.....)

In python indexes can be postitve or negative
+ve index means forward direction from left to right
-ve index means backward direction from right to left

Note: if we try to fetch value using the index more than the indexed position then we get IndexError: string out of range

Get Kohli from Virat Kohli





Starting is position where you want to cut the piece and it includes that position character
endpoisition is where you want to end the piece but ending index **chracter wont by inclusive**
If we dont give any endposition, ending index will be treated as the end of entire string

Note:
In python int, float, complex, bool and str are the fundamental datatypes

TYPE CASTING:
a = "10"
b = "20"

The process of converting one type to another type is nothing but a typecasting or type coersion.
These are the following functions for typecasting
1. int() :
We can use this function to convert values from other type to int

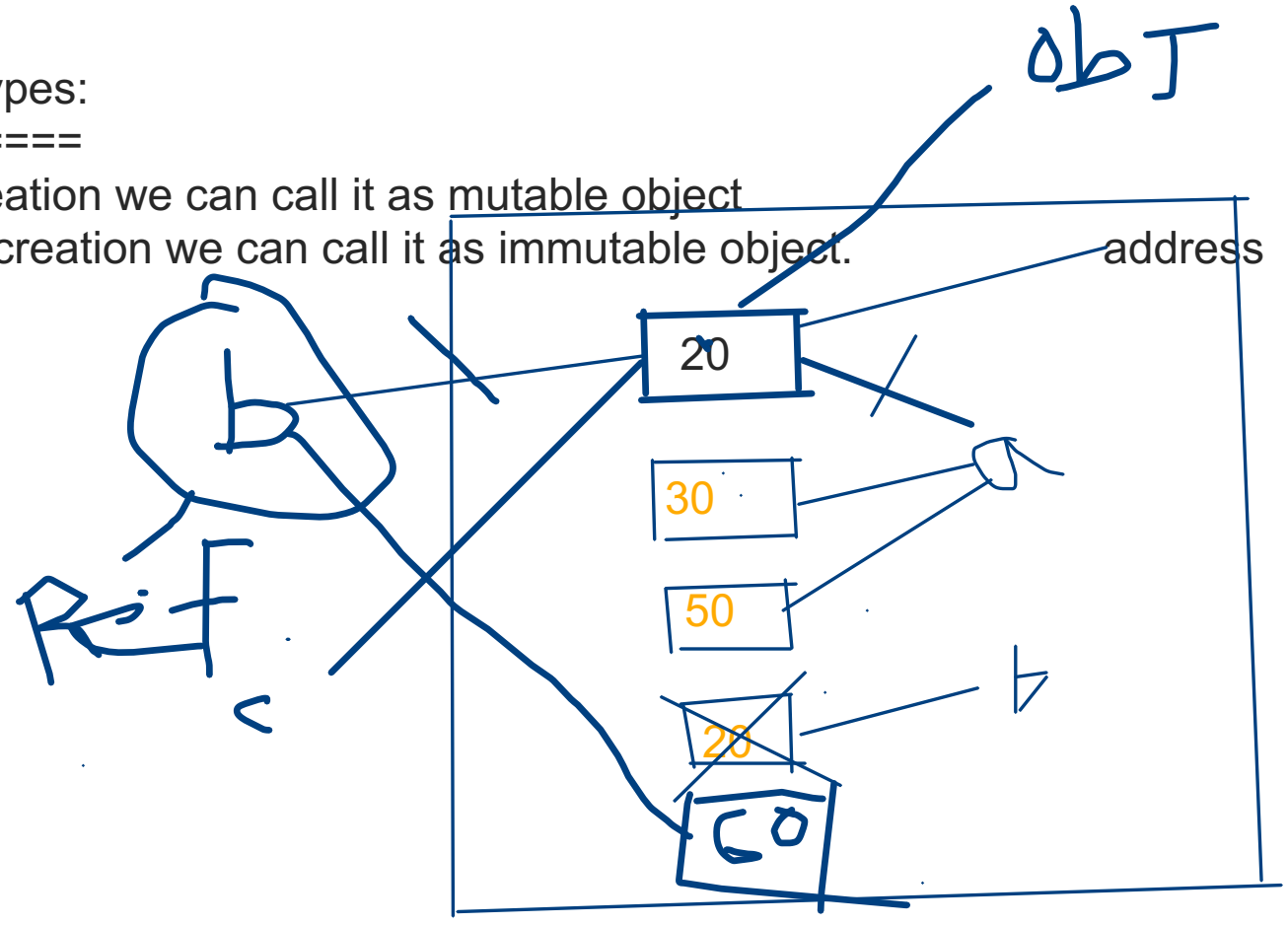
Note:
1. We can convert any type to int except complex type
2. If we want to convert string to int type, mandatorily string should have only integral values and that should have base-10, otherwise it will get ValueError

2. float() --> We can use float function to convert other types(int, bool, str, etcc) to float type
Note:
1. We can convert any type to float except complex type
2. If we want to convert string to float type, mandatorily string should have only floating or integral values and that should have base-10, otherwise it will get ValueError

3. complex() --> We can use this function to convert any other type to complext type

4. bool() --> We can use this function convert any toher type to bool
5. str() --> We use this function to convert any other type to string

Immutability w.r.to Fundamental DataTypes:
==========
If I'm able to change the object after creation we can call it as mutable object
If I'm unable to change the object after creation we can call it as immutable object.



In python if a new object is required, PVM wont create new object immmediately rather it will check the exist object, if any object has same content or data, it will reuse that object.

If existing objects are not having same content, then only it will create a new object

Advanced Datatype or datastructures:

bytes
bytearray
list
tuple
set
range
dict

bytes datattype :

bytes data type is helpful to represent a group of numbers, like a an array
the only allowed values are 0 to 255, accidentally if we give more than this range then we will get valueerror

Syntax:
we have to use bytes(x) --> x is having sequence of numbers
Ex: bytes([10,20,30,40])

bytes is immutable we can not change its values or reassign once it is created, if we try to modify we will get type error
if we want to store data within 0 to 255 and i dont want anyone to be able to modify/change after creation, then bytes is right choice

1, 2 3 4 5 6 7 8 9, 10, ... 23 --> state codes in vodafone

bytearray data type:
bytearray is exactly same as bytes datatype except that it is mutable(i.e we can change the values)

the only different between byte and bytearray is bytes is immutable and bytearray is mutable

list datatype:
If we want to store group of values in a single variable, where values having different type of data or same type of data

marks = [10,20,30,40] --> Same type of data, this is possible (homegenious data)
student_data = [12345, "kasi yedumati", 98, True] --> Different type of data(heterogenious data)

1. It maintains insertion order
2. it allows heterogenious(different type of values) data
3. duplicate data also allowed
4. it is growable in nature(we can add data after creation or remove data or modify data or fetch data)
5. we should create list variable values using [ ]
Ex: sutdent_list = ["kasi", "virat","dhoni"]

student_names = ["kasi", "bhuvan", "chandu", "varshith", "harika", "chandu"]

student_name[3] --> varshith

Note: List is an ordered, mutable, heterogenious collection of elements and also duplicated are ordered

tuple data type:

tuple is exactly same as list, except that it is immutable i.e we cannot change the values once it is created

syntax : we have to create a tuple using paranthesis ( )

Ex: (10,20,30)

Note: tuple is the read only version of list

range datatype:

range datatype represents sequence of numbers

rang is also immutable, the values/numbers present range datatype are not modifiable.

Syntax1:

range(x) --> x will be a number

range(20) --> it generates numbers from 0 to 19

Syntax 2:

range(start, end) --> start will be a number and end also should be a number

Syntax3:

range(start, end, increment)

start --> it is where i want to sequence range should start from

end --> it is where i want to stop range

increment --> how many values you have to increment for every generate sequence numbers

range(10, 20, 2)

Set datatype:
Whenever we want to store group of values where insertion order is not required and duplicated are not allowed, in that case we go for set

1. insertion order is not preserved
2. duplicates are not allowed
3. hetergenious objects are allowed (same or different type of values)
4. it doesnt follow indexing
5. it follows hashing mechanism to insert or update data or manage data in set
6. it is mutable (once the object is created we can modify)
7. it is growable in nature

fruits_set = {'banana', 'apple'}
fruits_set = set() --> create an empty set

frozenset Datatype:
frozeznset is exactly same as set , except that it is immutable.
Once it is created, we cant add, remove or modify its items.

Syntax:
frozenset([10,20,30])

dict datatype :
If we want to store a group of values as **key-value** pairs then we should go for dictionary datatype

Syntax:
contacts = {"name" : "mobilenumber"}

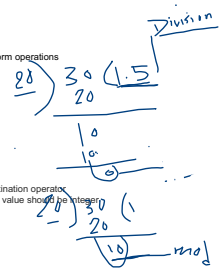
employee_data = ["kasi", 1234, 123000, "hyd"]   --> list data form
employee_data =
{
  "name" : "kasi",
  "employeeid" : 12345,
  "salary" : 12345.50,
  "location" : "hyd"
}

Key must be unique
value can be unique or duplicate
immutable datatype (string, tuple, frozenset)
Using key we can access the data

Values can be any datattype

if we try to add same key twice, it will just consider latest entry value or it overrides values within the same key
it wont throw any error

Datatype	Duplicates	Insertion order	heterogenious	mutable or immutable	syntax	follows index?
int	not applicable	not applicable	only integers 0 to 9	immutable	a = 10; b = 20	
float	not applicates	not applicable	only floating values	immutable	a= 10.5	
complex	not applicable	not applicable	real and imaginary	immutable	a = 3+0j	
bool	not applicable	not applicable	True, False	immutable	a = True ; a= False	
string	allowed	applicable	No	immutable	a= "hello"	Yes, eg: a[2]
bytes	allowed	applicable	No(0 to 255)	immutable	b = bytes([10,20,30]}	Yes
bytearray	allowed	applicable	No	mutable	b =bytearray([10,20]}	Yes
list	allowed	applicable	Yes	mutable	names=['shreyas','sai']	Yes, eg: names[1]
tuple	allowed	applicable	Yes	Immutable	x=(1,3+2j, 'chandu')	Yes, eg: x[2][2]
range	allowed	applicable	No	mutable	x=range(0,9); 0,1,2,3 4,5,6,7,8	Yes
set	not allowed	not preserved	Yes	mutable	x={1,2,3,4}	No (because it follows hashing)
frozenset	not allowed	not preserved	Yes	Immutable	x=frozenset([1,2,3]}	no
dict	duplicate keys are not allowed	applicables	Yes	Mutable	d={'a' : 1, 'b':2}	we access values with key



Operators:
Operator is a symbol or keyword using which we can perform operations

1. Arithmatic operators
+ --> Addition/sum values
- -> subtraction
* -> multiplication
/ -> Division operator
% --> modulo operator (it returns the reminder

// --> Floor division operator
** -> Power operator or exponential operator

+ operator we can against strings also, we call it as concatination operator
* operator also we can use against strings, but alteast one value should be integer
String multiplication operator

2. Relational or Comparision operators
>
<
>=
<=

Comparision operators will compare any two values and return boolean values True or False
Equality operators --> == !=

3. Logical operators
There are three logical operators: **and , or , not**
These keywords/operators we can apply on all datatypes

These operators also check and return **boolean value**

and --> If both arguments are **True** then only result is **True**
or --> If atleast one argument is **True** then output will be True
not --> complement(if we apply not for True it will become False, if we apply not for False it will return False

	A	result
not	True	False

	A	B	
and	True	True	True
	True	False	False
	False	True	False
	False	False	False

	A	B	Result
or	True	True	True
	True	False	True
	False	True	True
	False	False	False

0 means False
non-zero means True
empty string is always treated as False

if a is evaluates to False return a otherwise it returns y

4. Assignment operators
We can use assignment operator to assign value to a variable
Eg: x = 10
We can combine assignment operator with mathematical operators

x += 10

5. Ternary Operators
Syntax:
x = **firstValue** if **condition else secondValue**

if condition becomes True then firstValue will be returned, else(if condition becomes False) secondValue will be returned

5. Bitwise operators
6. Special operators
Python has two special category operators they are
6.1 : Identity Operators
We can use identity operators for address comparison
is
is not

Eg: a= 20
b = 30
a is b --> This returns True if both a and b are pointing to the same object, otherwise returns False
a is not b --> This returns True if both a and b are not pointing to same obj, otherwise returns False

Note: We can use is operator for address comparion whereas == operator for content comparion

6.2: Membership operators:
We can use membership operators to check whether the given object is present in the collection or datastructures
(String, List, Set, Tuple or Dict)

in --> Returns True if the given obj present in the specified collection
not in -> Returns True if the given object is not present in the collection

Bitwise Operators:
We can apply these operators bitwise
These operators we can apply only on int and boolean types, if we try to apply on any other type we will get Error

Eg: print(5&4) -->Valid
print(True & True) --> Valid
print("abc"& "xyz") --> Invalid (Error)
print(0.5&10.5) ---> Invalid(Error)

5 -->
4

&
|
^
~

<<
>>

bitwise compliment operators:
We have to apply complement for total bits(single value)

Eg: print(~5) --> -6

Operator	Description
&	If both bits are 1 then only result is 1 otherwise result is 0
|	If atleast one bit is 1 then result is 1 otherwise result is 0
^	if bits are different then result is 1 otherwise result is 0
~	bitwise complement operator i.e 1means 0 and 0 means 1
>>	Bitwise right shift operator> it swifts positions to the right, and result the final value
<<	bitwise left shift operator -> it swifts postiions to the right and result the final value

<< left shift operators
Eg: print(10<<2)

right shift >>
print(10>>2)

Python Comments:
Comments in python are used to explain the code and provide futuristic TODO items
these comments are ignore by python during program execution

Advantages:
1. Readability improvement
2. It can be used to identity the funcitonality or structure of the code

Single line comments :
Single line comments are starts with hashtag #
Ex:
this is a single comment

Multi-line comments:
We can apply hashtags multiple times
using a string literals using """comment"""
'''testcomment''

Input and Output Statements:
Static inputs/values --> If the value or input is always same, we call it as static value/input
Dynamic inputs/Values --> If the value or input keeps changing over the time, we call it as dynamic input

Whenever we are building applications or command line based scripts or automation scripts, we take the input from Either ui or command prompt or terminal

To take the dynamic inputs from the keyboard we have a function called **input()**

input():
it is used to take dynamic input from keyboard while running the program, and it will wait for user to enter the input
and start executing the program once the data is entered

input function always return data in string type, before perfomring any operation we have to do the typecasting if at all it is needed.

```
# reading multiple values at a time
# split() is the function in string, which will split values based on delimeter/saperator, if we dont specify any saperator
# by default it will saperate based on space(" ") and it will return splitted values in list type
# a = input("Enter 2 numbers in a space saperated:").split()
```

eval():

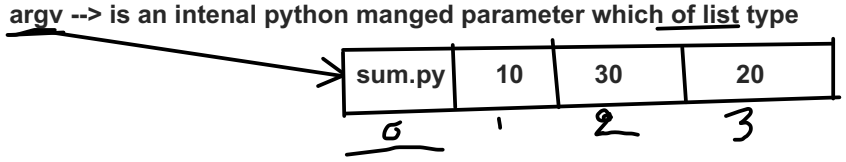
It will take a string as in put and evaluate the result

eval can evaluate the input to list, tuple, set, based on the provided input

Command Line Arguments:
The argument which are passing at the time of execution are called command line arguments

Eg: python3 sum.py 10 30 20
Command line arugments

Within in the python program these arguments are available in **argv, which is present in sys module**
argv --> is an intenal python manged parameter which of list type



Note1: By default all arguments will be treated as strings only
Note2: Within the python program command line args are available in string form based on our requirement, we need to convert them to required type
Note3: Usually space is saperator between command line args, if our command line arg itself has a space then we should enclose within the double quotes

Output statements:
We can use print() function to dislay output

Form-1: print() without any argument

Print Syntax:

print(*args, sep= ' ', end='\n', file=sys.stdout, flush= False)

Form3: print() with variable number of arguments

a,b,c= 10,20,30
print("numbers are", a,b,c)

by default sep is space, we can override by explicitly giving a saperator

print(a,b,c, sep='&')

end parameter is used end the current print statement, by default end will be \n which is new line

Form5: print with objects

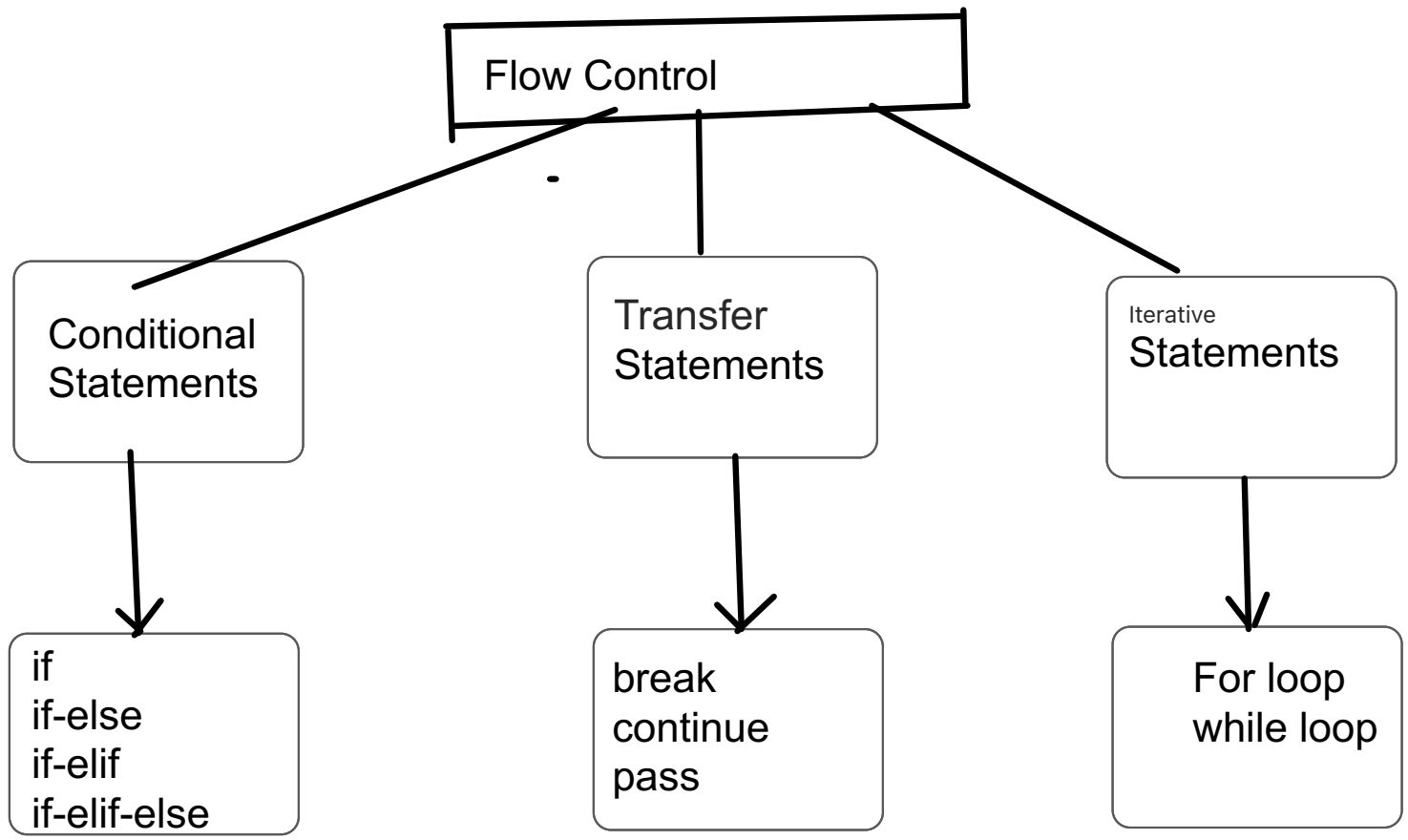
Form6: print(string, varfiable list)
we can use print stament with string and any number of arguments

Form7: print(formatted string)
%i -->int value
%d --> int value
%f --> float
%s --> string

Form8: print with replacement operator { }

Flow Control:

Flow control describers the order in which statements should be executed and also which statements to execute



Conditional statements:

Conditional stematements are such that, if your condition is met only then the statements will be executed.

aka, if we want to execute any statement(s) conditionally, then we go for conditional statement

if:

Syntax:

```
statement1
statement2
if condition:
   statement3
   stetement4
statement5
statement6
```

in this program statement3, 4 will be execution only when condition is True

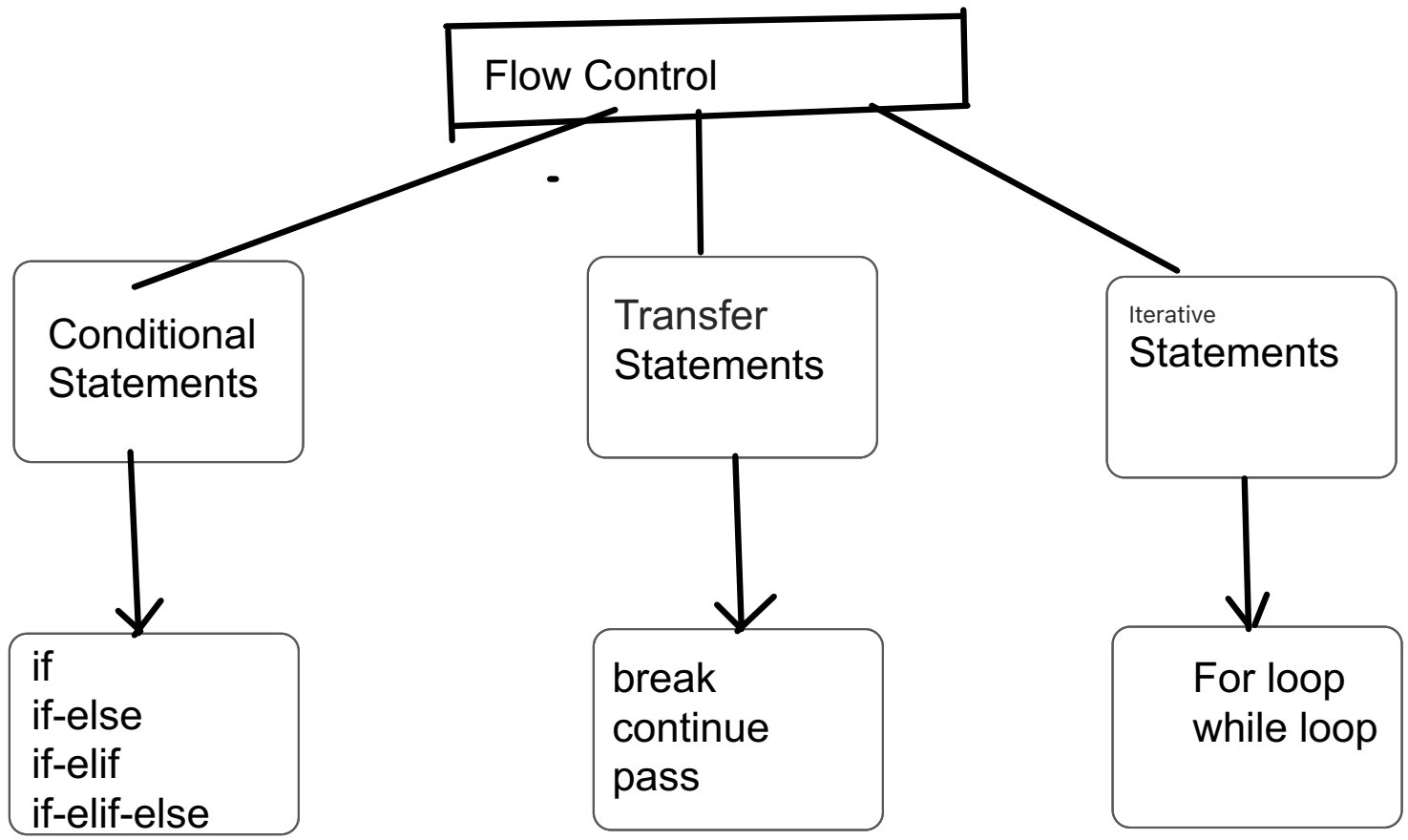
statement1,2,5,6 will execute, irrespective of condition is True or False

if block of statements

Note: At a time either if block statements or else block statments will execute not both

Flow Control:

Flow control describers the order in which statements should be executed and also which statements to execute



Conditional statements:

Conditional stematements are such that, if your condition is met only then the statements will be executed.

aka, if we want to execute any statement(s) conditionally, then we go for conditional statement

if:

Syntax:

```
statement1
statement2
if condition:
   statement3
   stetement4
statement5
statement6
```

in this program statement3, 4 will be execution only when condition is True

statement1,2,5,6 will execute, irrespective of condition is True or False

if block of statements

Note: At a time either if block statements or else block statments will execute not both

3. if-elif-else:
==========
Syntax:

```
if condition1:
   Action1
elif condition2:
  action2
elif condition3:
  action3
else:
  action4
```

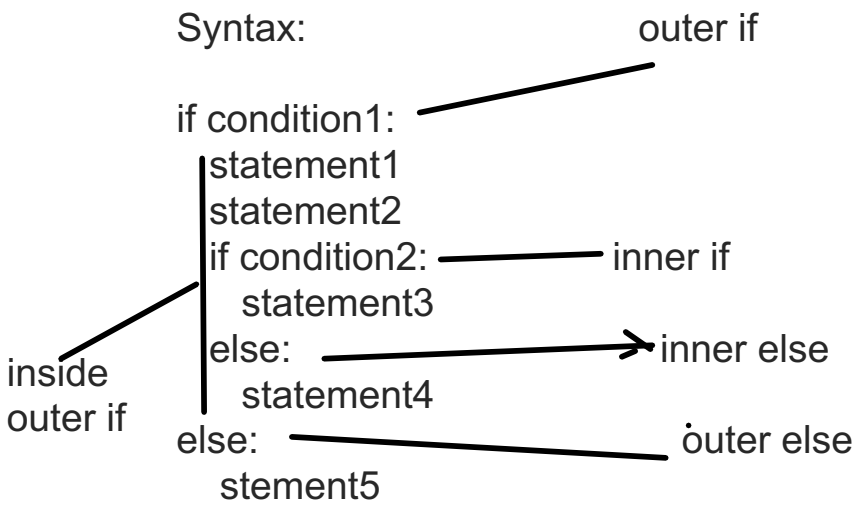
<u>At a time only one action will/has to be executed</u>, then we got if-elif-else

condition1 is True --> Action1 will be executed
condition2 is True -> action2 will be executed
condition3 is True --> action3 will be executed
if no condition is True then else block will execute

we dont have switch cases in python unlike other programming languages

nested if :
a if within another if or if withing another else

Syntax:



if condition1 is True all the statements including nested if-else will execute
if condition1 and nested if condition 2 is True then only statement3 will execute

Iterative/repetitive Statements:
If we want to execute a group of statements multiple times then we should go for iterative statements

Python supports 2types of iteratives
1. for loop
2. while loop

1. for loop:
If we want to execute some action for every element present in the some sequence(string, list, tuple, dict, range, set, frozenset, etc...)

Syntax:

```
for x in sequence:
    body(statements)
```

sequence --> string or any collection(list, set, tuple,etc.....)
x means each value in the sequence
body will be executed for every element in the given string

2) while loop:
If we want to execute a group of statements iteratively/repetitively until some condition is false, then we should go while loop

Syntax:

```
while condition:          group of statments that we want to execute when condition is True
    body
```

Nested loops:
Sometimes we can take a loop inside another loop, which we call it as nested loops.

```
for i in range(1,5):  ------ outer for loop
    for j in range(1,5):  --> inner for loop
        print(j)      --> inner for loop body
outer for loop body
```

```
Iter1:  i =1, j= 1 2 3 4
Iter2:  i=2, j = 1 2 3 4
Iter3:  i =3, j = 1 2 3 4
Iter4:  i =4, j = 1 2 3 4
Iter5: end the loop
```

Transfer statements:
1) break
We can use break statement inside loops to break the loop execution based on some condition

2) continue:
we can use continue statement to skip current iteration and continue the next iteration

3) pass:
pass is a keyword in python

In our python programming syntactically, if block is requires a block of statements.
but if we want to give empty block python will throw en error, using pass keyword we can create empty blocks

pass -> it is an empty statement
--> it is null statement

del statements:

del is a keyword in python

We can delete variables and objects which are pointing to mutalble objects, but we cannot delete immutable objects
name = "kasi"
del name --> valid
del name[0] --> not valid becuase "kasi" is immutable object

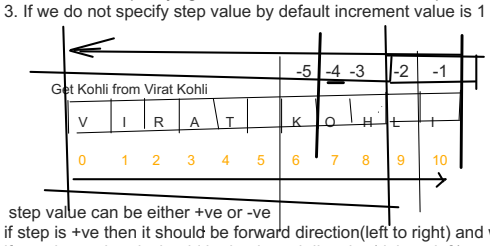
String DataType:

Accessing elements in the string:
1. using index value
2. using slicing operator

Syntax: s[beginIndex:endIndex:step]

beginIndex: From where we have to consider/cut the slice(substring)
endIndex: Where/when We have to terminate the slice(substring)
step --> incremented value while slicing

virat kohli
Note:
1. If we are not specifying beginIndex then it will consider from the begining of the string
2. If we are not specifying endIndex then it will consdier up to the end of the string
3. If we do not specify step value by default increment value is 1



a 6 : 10

step value can be either +ve or -ve
if step is +ve then it should be forward direction(left to right) and we have to consider to begin to end-1
if step is -ve then it should be backward direction(right to left) and we hvae to consider begin to end+1

len() is a pre-defined/built-in function to find bnumber of characters present in the string

Operators in string:
1. + operator for concatination
2. * operators for repition

membership operators also can be used to check existing of a substring or a character in the given string

Functions in string datatype:

Removing spaces from the string
1. rstrip() -> To remove spaces from right hand side
2. lstrip() -> To remove spaces from left side
3. strip() --> To remove spaces from both side

4. find() --> it returns the index of first occurance of the given string, if it is not available then we will get -1
Note: by default find() can search total string, we can also specify boundaries
find(substring, beginIndex, endIndex)

5. index() --> It is exactly same as find() except that it will throw ValueError if the given substring is not found
6. rindex()
7. rfind()
8. count(substring) -> it find the number of occurances of the given string, it will search through the entire string
count(substring, beginindex, endindex) --> it will search from begin index to endindex
9. replace(oldstring, newstring)
10. split(saperator) --> we can split the given string based on the delimeter/saperator, if we dont specify any saperator it will saperate the given string with space by default

saperator can be anything, it can be a symbol or it can be a substring also

s= "Learning python is very easy, python is open source programming language"

s.split("python")
it will return the list of the splitted string

["Learning ", "is very easy, ", "is open source programming language"]

11. join() --> we can join a group of strings(list or tuple) with respect to the given delimeter
students = ["kasi", "virat", "dhoni"] --> list type
this will returns the joined string

changing the case of a string:
12. upper() --> To covert all the characters to uppercase
13. lower() --> To convert all the characters to lowercase
14. swapcase()--> to convert all the lower case character to upper case and all upper case characters to lower case
15. title() --> To convert all the character to title case, first character of every word should be capital and other chars should be small
kasi yedumati --> Kasi Yedumati
16. capitalize() --> Only first character will be converted to upper case and all other characters in lower case

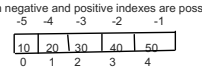
checking the starting and ending part of the string:
17. startswith(substring)
18. endswith(substring)

To check type of characters present in a string:
==========
1. isalnum() --> Returns True if all the characters are alphanumeric (a to z, A to Z and 0 to 9)
2. isalpha() --> Returns True if all characters are alphabets(a to z, A to Z)
3. isdigit() --> Returns True if the given string contains all the chacters are numbers only (0 to 9)
4. islower() --> Returns True if all the characters are in lower case (a to z)
5. isupper() --> Returns True if all the chactrers are in upper case(A to Z)
6. istitle() --> Returns True if the string is in title case(Kasi Yedumati, Virat Kohli)
7. isspace() --> Returns True if the string is containing only spaces

List Data Structure:
If we want to represent a group of individual objects or elements in a single variable or single entity where insertion is maintained and duplicates are allowed. then we should go for list.

insertion order is preserved
duplicated are allowed
heterogeniours value are allowed(different type of value)
list is dynamic --> we can add or remove items from the list
it is mutable

Python supports indexing for list, both negative and positive indexes are possible
[10, 20, 30]
["pizza", "burger", "coke"]
[123, "kasi", 99.5]



Creation of list objects:
1. l = [ ] --> Empty list
2. l = ["pizza", "chocolate", "coke"] --> create a list with items
3. using eval with input function, we can create a list
4. we can create list with list()
5. with split() function --> "kasi yedumati".split() --> ["kasi", "yedumati"]

Accessing elements of list:
we can access the elements of the list using index
we can access elements uisng slice operator
1. by using index:
==========
list follows both +ve and -ve indexing
+ve means from left to right
-ve index means right to left

2. by using slice operator
==========
list[startIndex: endIndex: step]

Traversing the elements in the list:
sequential access of each element in th list is called traversal

1. by using while loop

2. by using for loop

Functions of list:
1. len() --> return the number of elements in the list
2. count(item) --> it returns the number of occurance of specified item in the list
eg: list.count("pizza")

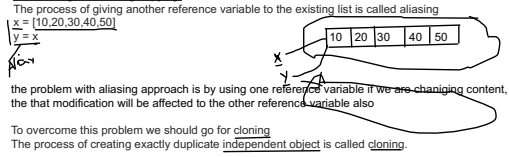
3. index() --> returns the index of first occurrance of the specified item
eg: list.index("pizza")
Note: if the specified item is not present we will get ValueError, before going to use index() we have to check whether it is present or not
manipulate/modify the existing list
4. append() --> used to append/add item in the end of the list
5. insert(position, item) --> to insert item at specified index position
6. extend() --> to add list of items at a time to existing list, or to add multiple items at a time
7. remove(item) --> used to remove item from the list, if multiple occurances are found then it will remove only first occurance
Note: if the given item is not found in the list, it will throw ValueError, before using remove fuction it is recommended to check if item is exist or not
8. pop() -> it removes and returns the last element of the list
pop(index) --> it removes and returns the indexed value in the list
Difference between pop and remove

remove()	pop()
1. to remove specified element	1. used to remove last element or specific index from list
2. It can't return any value	2. it returns removed value
3. if the specified element is not present, we get valueerror	3. if the specified index is not there, we get indexerror

Ordering elements of List:
9. reverse() --> to reverse items in the list
10. sort() -> used to sort elements in both ascedning order and descending order

Note: to use sort(), compulsorily list should contain only homegenious elements/data, otherwise we will get TypeError

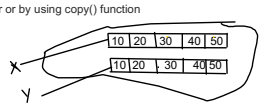
List aliasing and cloning/copying:
The process of giving another reference variable to the existing list is called aliasing



the problem with aliasing approach is by using one reference variable if we are chaniging content, the that modification will be affected to the other reference variable also

To overcome this problem we should go for cloning
The process of creating exactly duplicate independent object is called cloning.

we can implement closing by using slice operator or by using copy() function



Using mathematical operators for list objects:
We can use + and * operators on list objects

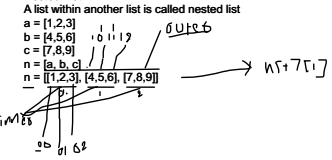
Note: to use + operator complusoriliy both arguments should be list objectss, otherwise we get TypeError

Comparint list objects:
Whenever we are using comparision operation(==, !=) for list objects the following order should be considered

1. number of elements
2. order of elements
3. content of elements including case

Membership operators:
used to check whether element is there or not in the list
in
not in

Nested List:
A list within another list is called nested list
a = [1,2,3]
b = [4,5,6]
c = [7,8,9]
n = [a, b, c]
n = [[1,2,3], [4,5,6], [7,8,9]]



List comprehension:
it is very easy and compact of creating list objects from any itrable(list, tuple, dict, range, string) based on condition operation/outcome

Syntax:
list = [expression for item in list if condition]